

Application-aware Stateful Firewalls (Network Firewalls Part II)



ICT2203 Networks Security
LAB 7 NOTES

LAB 7

- ➡ Configure **MPF inspection security policy** in ASA to inspect **application layer protocols** such as **HTTP**

To understand **application-aware** stateful firewall, we'll now briefly discuss the configuration of **application inspection** in ASA

In general, configuring **application inspection** involves **3 steps** as follows:

Step 1: Configure **application** layer protocol **inspection** **class map**

```
host(config)# class-map type inspect application [match-all | match-any]  
              application_class_map
```

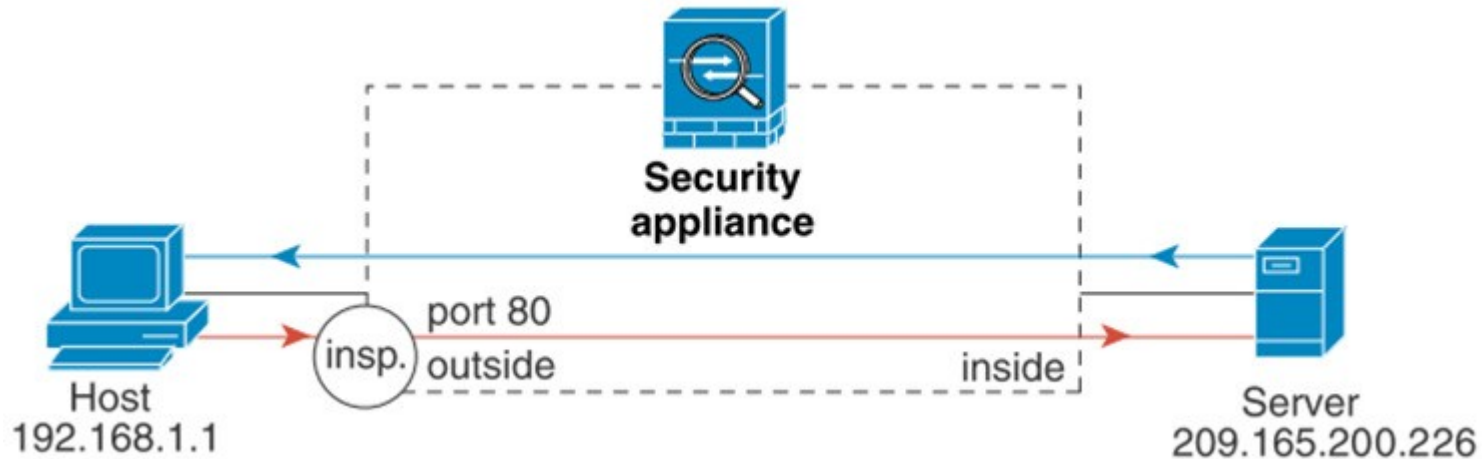
Step 2: Configure **application** layer protocol **inspection** **policy map**

```
host(config)# policy-map type inspect application application_policy_map
```

Step 3: Apply **inspection** **policy map** over **MPF** layer 3/4 **policy map** discussed in previous slides.

```
host(config)# policy-map policy_map_name  
host(config-pmap)# class class_map_name  
host(config-pmap-c)# inspect application application_policy_map
```

There are too many different commands specific for inspecting different applications. To be specific, we'll focus on configuring **HTTP inspection policy map** in this module.



```
ciscoasa(config)# class-map type inspect http [match-all | match-any] cmap_name
ciscoasa(config-cmap)# match [not] {request | response | req-resp} ...
ciscoasa(config-cmap)# ...
ciscoasa(config-cmap)# exit
ciscoasa(config)# policy-map type inspect http pmap_name
ciscoasa(config-pmap)# class cmap_name
ciscoasa(config-pmap-c)# {drop-connection [log] | log | reset [log]}
ciscoasa(config-pmap-c)# exit
```

STEP 1

HTTP inspection
policy map can be
configured using
these **match**
commands

```
ciscoasa(config-cmap)# match [not] request header {[field]
[regex [regex_name | class regex_class_name]] |
[length gt max_length_bytes | count gt max_count_bytes]}

ciscoasa(config-cmap)# match [not] request method {[method] |
[regex [regex_name | class regex_class_name]]}

ciscoasa(config-cmap)# match [not] request uri {regex [regex_name | class
regex_class_name] | length gt max_bytes}

ciscoasa(config-cmap)# match [not] request args regex [regex_name | class
regex_class_name]

ciscoasa(config-cmap)# match [not] request body {regex [regex_name | class
regex_class_name] | length gt max_bytes}

ciscoasa(config-cmap)# match [not] req-resp content-type mismatch

ciscoasa(config-cmap)# match [not] response header {[field]
[regex [regex_name | class regex_class_name]] |
[length gt max_length_bytes | count gt max_count]}

ciscoasa(config-cmap)# match [not] response status-line {regex [regex_name | class
regex_class_name]}

ciscoasa(config-cmap)# match [not] response body {[active-x] | [java-applet] |
[regex [regex_name | class regex_class_name]] | length gt max_bytes}
```

The list of **regular expression metacharacters** and their usages are as follows:



Metacharacter	Name	Function
.	Dot	Matches any single character. Example: b.d matches bad, bbd, bcd, bdd, bed, and so on
()	Subexpression	Groups the characters inside the parentheses as a single expression for matching with other metacharacters.
	Or	Matches either expression that separates. Example: com net matches whatever.com or whatever.net Example: Ma(r y) matches Mar or May
?	Question mark	Matches 0 or 1 of the expression just before the ?. Example: e?smtp matches smtp (zero e's) or esmtp (1 e) Example: (12)? matches 4444, 12444, 1212444, and so on
*	Asterisk	Matches 0, 1, or any number of the expression just before the *. Example: w* matches cisco.com and www.cisco.com
+	Plus	Matches at least one of the expressions just before the +. Example: w+ matches www.cisco.com, but not cisco.com

Regular expression metacharacters (continued)

Metacharacter	Name	Function
{n}	Repeat	Matches if the expression just before {n} is repeated exactly n times. Example: (test){2} matches testtest but not testtesttest
{n,}	Minimum repeat	Matches if the expression just before {n,} is repeated at least n times. Example: (test){2,} matches testtest and also testtesttest
[abc]	Character class	Matches any of the characters listed between the square brackets. Example: [dfhl]og matches dog, fog, hog, and log, but not frog
[^abc]	Not character class	Matches any character that is not listed between the brackets. Example: [^dfhl]og matches cog, but not dog, fog, hog, or log
[a-c]	Character range class	Matches any character in the range from a to c. Example: [a-z] matches any lowercase letter, [A-Z] matches any uppercase letter, [0-9] matches any digit

STEP 2

Configure **HTTP inspection policy map** to specify **actions** to be performed on matched traffic

```
ciscoasa(config-pmap-c)# {[drop [send-protocol-error] |  
drop-connection [send-protocol-error] | mask | reset] [log] |  
rate-limit message_rate}
```

- **drop** keyword drops the packets
- **send-protocol-error** keyword sends a protocol error message
- **drop-connection** keyword drops the packet and closes the connection
- **mask** keyword masks out the matching portion of the packet
- **reset** keyword drops the packet, closes the connection, and sends a TCP reset
- **log** keyword sends a system log message
- **rate-limit** *message_rate* argument limits the rate of messages

Note that not all actions are allowed for different match command.
Use CLI ? help for exact options available.

In addition, **HTTP inspection policy map** has special **parameters** option for specifying **action** to take if **HTTP protocol violation** occurs

To enter into **parameters** configuration mode in inspection policy map:

```
ciscoasa(config-pmap) # parameters  
ciscoasa(config-pmap-p) #
```

To configure **action** for **HTTP protocol violation**, enter the command:

```
ciscoasa(config-pmap-p) # protocol-violation [action  
                        [drop-connection / reset / log]]
```

To summarise, here is an example of **HTTP inspection class map** and **inspection policy map**.

```
ciscoasa(config)# regex url_example example\.com
ciscoasa(config)# regex url_example2 example2\.com
ciscoasa(config)# class-map type regex match-any URLs
ciscoasa(config-cmap)# match regex url_example
ciscoasa(config-cmap)# match regex url_example2

ciscoasa(config)# class-map type inspect http match-all http-traffic
ciscoasa(config-cmap)# match req-resp content-type mismatch
ciscoasa(config-cmap)# match request body length gt 1000
ciscoasa(config-cmap)# match not request uri regex class URLs

ciscoasa(config)# policy-map type inspect http http-map1
ciscoasa(config-pmap)# class http-traffic
ciscoasa(config-pmap-c)# drop-connection log
ciscoasa(config-pmap-c)# match req-resp content-type mismatch
ciscoasa(config-pmap-c)# reset log
ciscoasa(config-pmap)# parameters
ciscoasa(config-pmap-p)# protocol-violation action log
```

STEP 3

Finally, apply the **HTTP inspection policy map** into the MPF layer 3/4 policy map for it to take effect

```
ciscoasa(config)# policy-map test
ciscoasa(config-pmap)# class test
ciscoasa(config-pmap-c)# inspect http http-map1
```

```
ciscoasa(config)# service-policy test interface outside
```

Note: If you modify an in-use HTTP inspection policy map, you must remove and reapply it for the changes to take effect.

```
ciscoasa(config)# policy-map test
ciscoasa(config-pmap)# class http
ciscoasa(config-pmap-c)# no inspect http http-map1
ciscoasa(config-pmap-c)# inspect http http-map1
```